

Tushar's Workspace

What it is, what problem it solves, and how it works (technical name: *claude-queue*)

In one line: a small program that runs all the time on a Mac. It keeps a to-do list of coding jobs for different projects and has **Claude Code** (an AI coding assistant) work through them one at a time, with nobody sitting at the computer. Jobs can be added and checked from a phone.

1. The problem it solves

Normally, Claude Code only works while someone sits at the laptop, types a request and waits. That causes a few problems:

- 1 **You have to be at the computer.** You can't hand it work from your phone while you're out.
- 2 **You have to babysit it.** When a job finishes, nothing happens until someone types the next one.
- 3 **Usage limits stop the work.** When Claude hits its usage limit, it stops, and someone has to remember to start it again later.
- 4 **It needs permission for some actions.** Some commands, like deploying, need a "yes" from a person. Without a way to answer, the work just gets stuck.
- 5 **It's hard to track.** With several projects on the go, it's hard to see what ran, what worked and what failed.

2. How it solves them

Problem	Solution
Must be at the computer	A web dashboard that also works on a phone, reached securely through Tailscale
Babysitting	A queue: when one job finishes, the next one starts on its own
Usage limits	It notices the limit, waits until it resets, then carries on by itself
Permission prompts	It sends the request to the phone, and the owner taps Approve or Deny
Tracking	Every job is saved with its status, result and timestamps, and you can search them

3. A simple way to picture it

Think of a **restaurant kitchen with one chef**:

- **Projects** are the different restaurants the chef cooks for.
- **Tasks** are the order tickets.
- **The queue** is the ticket rail. Urgent tickets go first, then the oldest.
- **Octopus** is the chef. It cooks one order at a time, but it can work in any kitchen. That's where the name comes from: one brain with many arms.
- **The dashboard** is the screen where you place orders and watch their progress.

4. How a job flows through the system

- 1 A task is added to a project from the dashboard, for example *"fix the login bug"*.
- 2 It's saved in a small database with the status **queued**.
- 3 Octopus checks for new work every 10 seconds and picks the next task, highest priority first.
- 4 It runs Claude Code inside that project's folder with the request.
- 5 Claude does the work: it reads code, edits files and runs commands.
- 6 The result is saved and the task is marked **done** or **needs review**.
- 7 The phone gets a notification (optional, through ntfy).
- 8 Octopus moves straight on to the next task.

5. Main features

Feature	What it does
Projects	Each code folder is registered once with a name and a path. A project can be paused so its tasks don't run.
Task queue with priority	Jobs run one at a time, highest priority first, then oldest first. It runs one job at a time on purpose: there's only one Claude usage allowance, so running jobs in parallel wouldn't be faster.
Automatic resume after usage limits	If Claude says it has hit its limit, the task is marked rate_limited . The system tries to read the reset time from the message. If it can't, it waits 1 hour, then 2 hours, and so on, up to 6 hours. Then it tries again on its own.
Chats and replies	Each task opens like a chat. A reply, for example <i>"undo that last change"</i> , continues the same conversation, so Claude remembers what it just did. A brand-new task starts with a blank memory.
Quick-action buttons	One-tap replies inside a chat: <i>Status</i> , <i>Run tests</i> , <i>Deploy</i> , <i>Pull latest</i> .
Todos tab	A backlog for each project. Write ideas down, put them in order and tick the ones you want. Nothing runs until Run selected is pressed. The ticked items then become real tasks and run in the order you arranged them.
Approvals	Claude can edit files freely, but terminal commands need a human "yes". When Claude wants to run one, the dashboard shows an Approval needed card and the phone gets a notification. Tap Approve or Deny . If nobody answers within 1 hour, the request is denied automatically so the rest of the queue doesn't get stuck.
Safety limits	Any single task that runs longer than 1 hour is stopped. Queued tasks can be cancelled.
Dashboard	A web page that works on mobile and refreshes every 5 seconds. It shows what's running, what's next, the number of tasks in each status, and a searchable, filterable history.

6. Task statuses

Status	Meaning	Status	Meaning
queued	Waiting its turn	needs_review	Something went wrong, a person should look
running	Claude is working on it right now	rate_limited	Paused by the usage limit, resumes automatically
done	Finished successfully	cancelled	Stopped before it ran

7. Access and security

- It runs on the Mac at `http://127.0.0.1:8787`, and the dashboard is at `/dashboard`.
- **Tailscale** is a private network that links only the owner's own devices, which is how the phone reaches the Mac. The system isn't open to the public internet.
- An **API key**, a secret password, is required for every request. It stops even other devices on that private network from using the system unless they have the key. The dashboard asks for it once and remembers it.
- **Risk to be aware of:** Claude edits files without asking. So it's only pointed at projects stored in git, where any bad change can be undone easily.

8. The parts (for technical listeners)

File	What it does
<code>main.py</code>	Starts everything: the web API and Octopus together in one program
<code>worker.py</code>	Octopus: picks tasks, runs Claude Code, handles limits and notifications
<code>api.py</code>	The web API (FastAPI) that the dashboard and phone talk to
<code>db.py</code>	The database layer (SQLite, a single file called <code>queue.db</code>) that stores projects, tasks, todos and approvals
<code>approval_server.py</code>	Receives Claude's permission requests and waits for Approve or Deny
<code>dashboard.html</code>	The web page you see
<code>setup/...plist</code>	macOS settings (launchd) that start it at login and restart it if it crashes

Built with: Python, FastAPI, SQLite, the Claude Code command-line tool, macOS launchd, Tailscale and ntfy (for phone notifications).

9. Checking that it's alive

- `launchctl list | grep claudequeue` shows whether the service is running.
- `/queue/status` shows what's running and what's next.
- Logs are in `~/Library/Logs/claude-queue/`.

10. Not built yet

- iOS Shortcuts that call the system directly. These are planned; the dashboard already works on a phone.
- Priority rules smarter than "priority number, then oldest first".